Vamos destrinchar o funcionamento do código, focando na mudança de mentalidade entre a abordagem Procedural e a Orientada a Objetos (POO). A grande diferença entre as duas não é o que o programa faz (o resultado final é o mesmo), mas sim **como ele organiza as informações e as responsabilidades**.

Aqui está o detalhamento de cada parte conceptual do script:

1. O Armazenamento das Informações (Dados vs. Estado)

No Procedural: As informações do aluno (nome, frequência, notas) ficavam soltas na memória ou agrupadas num dicionário genérico (ex: `aluno = {"nome": "Carlos", "notas": [...]}`). O programa não tinha uma definição rigorosa do que era um "aluno".

Na POO (A Classe `Aluno` e o `__init__`): Nós criamos um molde oficial. O método `__init__(self, nome, frequencia)` é o construtor. O `self` representa a própria pessoa (o objeto). Quando fazemos `self.nome = nome` e `self.boletim = {}`, estamos a dizer: "Todo o aluno que existir neste sistema terá, obrigatoriamente, um nome, uma frequência e o seu próprio boletim". O dado deixa de ser uma variável solta e passa a ser o **estado** do objeto.

2. A Execução das Ações (Funções isoladas vs. Métodos do Objeto)

No Procedural: Tínhamos funções como `calcular_media(notas)`. A função era "cega" — ela não sabia de quem eram aquelas notas, apenas recebia uma lista, calculava e devolvia o valor. Você tinha de carregar os dados de um lado para o outro.

Na POO (O uso do `self` nos métodos): As funções passam a viver dentro do molde e chamam-se **métodos**. Repare no método `calcular_media(self, materia)`. Nós já não passamos a lista de notas para ele. Como ele tem o `self`, o próprio método olha para dentro do objeto, procura em `self.boletim` e faz o cálculo. O objeto ganha **comportamento**; o aluno "sabe" calcular a sua própria média.

3. Separação de Responsabilidades (Mistura vs. Encapsulamento)

No Procedural: As funções que faziam cálculos matemáticos (regras de negócio) estavam misturadas com as funções que pediam `input()` ao utilizador na mesma estrutura plana.

Na POO (A Classe `SistemaCadastro`): Aplicamos o princípio do **Encapsulamento**. Separamos quem "é" a entidade (o `Aluno`) de quem "opera" o programa (o `SistemaCadastro`).

Usamos `@staticmethod` (métodos estáticos) para funções como `ler_frequencia()`, que são apenas utilitários e não precisam de criar um objeto para funcionar.

O `SistemaCadastro` tem o papel exclusivo de interagir com o teclado, validar os números e, no final, **instanciar** (criar) um Aluno novo e limpo, passando-lhe os dados. O Aluno nunca interage com o teclado; ele só guarda dados e aplica regras de aprovação.

4. O Fluxo Principal (A Execução Final)

No Procedural: O foco da programação procedural é a **ação** (o verbo). No final do script, você executava `exibir_boletim(dados_aluno)`. A ação "engolia" o dado de forma passiva.

Na POO: O foco é o **objeto** (o substantivo). No bloco `if __name__ == "__main__":`, nós criamos a instância `aluno_cadastrado` e depois executamos `aluno_cadastrado.exibir_boletim()`. A leitura inverte-se. Você não manda o sistema exibir os dados do aluno; você "pede" ao próprio aluno que exiba o seu boletim. É uma modelagem muito mais próxima da forma como pensamos no mundo real.